INF1103

Team Project Framework

Phase 1 — Procedural Foundation

This document defines the project architecture, hard constraints, four mandatory manager modules, and deliverables for Phase 1. The domain and problem statement are chosen by the team. The architecture is fixed.

1. Overview

Every team — regardless of domain — builds the same four-module architecture. This is intentional: it allows consistent grading, clear role assignment, and a natural refactor path into object-oriented design in Phase 2.

The four modules form a data pipeline:

Pipeline: User / file → io_manager → ai_manager → logic_manager → data_manager

io_manager.py — collects input, displays output
ai_manager.py — sends every record through the AI API
logic_manager.py — applies business rules to AI output
data_manager.py — persists and retrieves records

Teams must choose a real-world problem domain and submit a one-paragraph problem statement for instructor sign-off by the end of Week 2. First-come, first-served within a cohort — no two teams may build the same application.

2. Procedural Foundation — What You Must Build

Phase 1 is a fully procedural Python application. It must have three functional layers corresponding to three of the four managers, with the logic manager connecting the AI and data layers.

2.1 Input layer (io_manager.py)

Collects structured user input via the terminal. All print() and input() calls in the entire codebase live here and nowhere else.

- Prompt the user for structured input using input()
- Validate type, range, and required fields — reject and re-prompt on bad data, never crash
- Return a clean, typed dict ready for ai_manager — no raw strings leaving this file
- All display functions (display_record, display_list, display_result) also live here

2.2 AI processing layer (ai_manager.py)

The core engine. Every record the system processes must pass through this module — no exceptions. If removing this file does not break the application's main purpose, the AI has been used incorrectly.

- build_prompt(record) — construct a clear, specific prompt from the record dict; instruct the model to return JSON
- call_api(prompt) — send to the AI API; catch connection errors and timeouts; log, do not crash
- parse_response(raw) — extract and parse the JSON from the response string; handle unexpected formats
- validate_response(data) — check schema: required keys, correct types, values in range; reject if malformed
- No domain logic in this file — only API interaction and response validation

2.3 Logic layer (logic_manager.py)

The domain brain. Acts on what the AI returned. This is where the team's chosen problem domain comes to life. Business rules here are specific to the application — and will differ significantly between teams.

- evaluate(record) — runs all business rules against the AI-enriched record; returns a decision dict
- score(record) — produces a numeric score from AI output fields; used for ranking or thresholding
- route(record) — assigns the record to a category, queue, or outcome path based on the evaluation
- At least one multi-condition rule that uses two or more fields from the AI response
- This is the richest module for Phase 2 refactoring — design it with that in mind

2.4 Data layer (data_manager.py)

The system's memory. All records are written to and read from flat files. The program must produce the same output across separate runs on the same input.

- save(record) — append a processed record to the CSV or JSON file after logic_manager has evaluated it
- load() — read and return all records on startup; return empty list if file does not exist; never crash
- query(filter_fn) — accept a filter function and return matching records; enables searching by field
- Handle missing or corrupt files without crashing: log the error, return empty list, continue

3. Hard Constraints

The following six constraints are non-negotiable. The first two marked ⚠ are instant-fail conditions — violating either results in an automatic competency failure for C1 regardless of other work.

#	Constraint	Detail	Impact
C1	**100% procedural — no classes**	No class definitions anywhere in the codebase. Assessors will grep. Functions only.	**⚠ Instant fail**
C2	**AI is the core engine**	Every record must pass through the API. Removing the call must break the app's main purpose.	**⚠ Instant fail**
C3	**Structured API responses**	Prompt must request JSON. Code must validate schema before using any field.	Grade impact
C4	**Flat file persistence**	All data saved to and loaded from CSV or JSON. Same output across separate runs.	Grade impact
C5	**Runs in Docker**	docker run on submission branch must produce working output on any machine.	Grade impact
C6	**Meaningful Git history**	Granular commits mapping to the lifecycle. Monolithic dumps are penalised.	Grade impact

⚠ On the AI constraint

The test: if you deleted ai_manager.py and the application still works, you have used AI as a feature, not the core engine.

Correct use: every record flows through the API; the response drives the logic in logic_manager.
Incorrect use: one optional call to 'enrich' a record, or a single call at startup to load a config.

Prompt design is graded. Vague prompts that return unparseable free text are a design failure, not bad luck.

4. Manager Reference

Quick-reference summary of all four manager modules, their required functions, and their Phase 2 refactor targets.

File	Responsibility	Must implement	Phase 2 becomes
`io_manager.py`	All user-facing interaction — input collection and output display	• Collect & validate input • All print() calls live here • Return clean typed dict • Display records & summaries	IOManager base. InputHandler and OutputHandler subclasses.
`ai_manager.py`	Core engine — every record passes through the AI API	• build_prompt(record) • call_api(prompt) • parse_response(raw) • validate_response(data)	AIManager with process(). Subclasses override build_prompt() for different record types.
`logic_manager.py`	Domain brain — applies business rules to AI output	• evaluate(record) • score(record) • route(record) • At least one multi-condition rule	Richest refactor. Rules → methods. Record types → subclasses with overridden evaluate().
`data_manager.py`	Persistence — stores and retrieves records across runs	• save(record) to CSV/JSON • load() on startup • query(filter_fn) • Graceful file error handling	DataManager with save(), load(), query(). Records become typed classes.

5. Phase 1 Deliverables

Five items are required for Phase 1 submission, plus the topic sign-off which must happen before Week 3.

5.1 Procedural source code

A working CLI application with the four manager files. Assessment criteria:

- 100% procedural — no class definitions in the codebase
- All four manager files present and functional
- Correct pipeline: io_manager → ai_manager → logic_manager → data_manager
- All print() and input() calls confined to io_manager.py (greppable)

5.2 Engineering report (maximum 5 pages)

Two required components on a single page:

→ Data flow diagram — showing how data changes state from user input → AI payload → file storage

→ Exception handling matrix — how the script handles: API connection failure, malformed API response, corrupt or missing data file, invalid user input

Hand-drawn diagrams are acceptable. The report must exist and contain both components.

5.3 Automated test script

A test script that verifies the core logic functions independently of the live API.

- Uses assert statements or a local testing tool
- Tests must cover logic_manager functions with hardcoded sample AI responses
- Must run without a live API connection
- Must pass cleanly inside the Docker container

5.4 Git repository

The submission repository must show evidence of iterative development.

- Granular commit history — not a single large commit
- Commit messages must be descriptive (e.g. 'add validate_response with schema check', not 'update')

5.5 Docker container

The application must run inside a Docker container.

- Dockerfile provided in the repository root
- docker run on the submission branch must produce working output
- All team members must verify it runs on their machine before submission
- No manual dependency installation should be required outside the container

5.6 Topic sign-off (due end of Week 2)

Before starting, submit a one-paragraph problem statement covering:

→ What real-world problem the application addresses

→ What data flows through the system (what the user inputs, what the AI processes, what gets stored)

→ What business rules logic_manager will apply to AI output